Array vs ArrayList vs LinkedList

A practical comparison in Java - Data Structures notes

1. Quick Overview

All three store a sequence of elements, but they are built differently in memory. That difference decides how fast each operation is.

	Array	ArrayList	LinkedList
What is it?	Built-in fixed-size block of memory	Resizable array (java.util)	Chain of nodes (java.util), doubly linked
Size	Fixed when created	Grows and shrinks automatically	Grows and shrinks automatically
Memory layout	Contiguous	Contiguous (internal array)	Scattered nodes linked by references
Element types	Primitives and objects	Objects only (wrappers like Integer)	Objects only (wrappers like Integer)
Length / size	`arr.length`	`list.size()`	`list.size()`

2. Array

An array is a fixed-size block of memory where all elements sit next to each other. Because of that, the computer can jump straight to any index.

- **Pros:** fastest access by index, lowest memory use, can store primitives (`int`, `double`...).
- **Cons:** size cannot change after creation, no built-in add/remove methods, inserting or deleting in the middle means shifting elements by hand.

```
int[] numbers = new int[3];     // size fixed at 3
numbers[0] = 10;
numbers[1] = 20;
numbers[2] = 30;
System.out.println(numbers[1]);       // 20
System.out.println(numbers.length);   // 3
// numbers[3] = 40;  -> ArrayIndexOutOfBoundsException
```

3. ArrayList

ArrayList is a class that wraps an array and handles resizing for you. When the internal array is full, it creates a bigger one (about 1.5 times larger) and copies the elements over.

- **Pros:** dynamic size, fast access by index, many ready methods (`add, remove, contains, indexOf`), great for reading.
- **Cons:** inserting or removing in the middle or at the front is slow (elements must shift), resizing costs a copy now and then, no primitives.

```
ArrayList<String> names = new ArrayList<>();
names.add("Ali");
names.add("Sara");
names.add(1, "Omar");           // insert at index 1 (shifts Sara)
System.out.println(names.get(0));    // Ali
names.remove("Sara");
System.out.println(names);           // [Ali, Omar]
```

4. LinkedList

Java's LinkedList is a **doubly linked list**: every node stores its data, a reference to the next node, and a reference to the previous node. Nodes can be anywhere in memory.

```
null <- [A] <-> [B] <-> [C] -> null
         ^ head           ^ tail
```

- **Pros:** adding or removing at the beginning or the end is fast, no shifting of elements, no resizing.
- **Cons:** no direct access by index (`get(i)` walks node by node), more memory per element (two extra references per node), slower to traverse because nodes are scattered.

```
LinkedList<Integer> list = new LinkedList<>();
list.add(10);
list.addFirst(5);               // fast: no shifting
list.addLast(20);               // fast
System.out.println(list);            // [5, 10, 20]
list.removeFirst();                  // fast
System.out.println(list.get(1));     // 20 (walks through nodes)
```

Note: the `MyLinkedList` we built by hand is *singly* linked (only a `next` pointer), so its `removeLast()` is O(n). In Java's doubly linked version, `removeLast()` is O(1).

5. Time Complexity Comparison

Operation	Array	ArrayList	LinkedList
Access by index `get(i)`	O(1)	O(1)	O(n)
Add at the end	Not possible (fixed size)	O(1) amortized	O(1)
Add at the beginning	O(n) manual shifting	O(n)	O(1)
Add / remove in the middle	O(n) manual shifting	O(n)	O(n) to find the spot, then O(1) to link
Remove first	O(n) manual shifting	O(n)	O(1)
Remove last	O(1) if you track the length yourself	O(1)	O(1)
Search by value	O(n)	O(n)	O(n)
Memory per element	Lowest	Low	Highest (extra references)

Amortized O(1) means that most adds are instant, and an occasional resize costs O(n), but the average stays constant.

6. Which One Should I Use?

Situation	Best choice
Fixed number of elements known in advance, or you need primitives and top speed	Array
Mostly reading elements by index, adding at the end, general purpose use	ArrayList
Many insertions and deletions at the beginning (queue, stack, deque behavior)	LinkedList
Not sure which one to pick	ArrayList (the usual default)

7. Same Task, Three Ways

Storing three numbers and printing the second one:

```
// Array
int[] a = {1, 2, 3};
System.out.println(a[1]);

// ArrayList
ArrayList<Integer> al = new ArrayList<>();
al.add(1); al.add(2); al.add(3);
System.out.println(al.get(1));

// LinkedList
LinkedList<Integer> ll = new LinkedList<>();
ll.add(1); ll.add(2); ll.add(3);
System.out.println(ll.get(1));
```

All three print 2, but behind the scenes the array and ArrayList jump straight to index 1, while the LinkedList walks from the head to reach it.

8. Summary

- **Array:** fixed size, fastest, supports primitives.
- **ArrayList:** flexible array, best for fast reading by index. The default choice.
- **LinkedList:** best when you constantly add or remove at the ends. Slow to access by index.
- Access by index: Array and ArrayList are O(1), LinkedList is O(n).
- Insert or delete at the front: LinkedList is O(1), ArrayList is O(n).